



RESEARCH ARTICLE

CodeNote.ai: A GitHub-Integrated Note-Taking and Automated Documentation System for Developers

Satyam Singh^{1*}, Vandana Sharma², Dhawal Beohar³

^{1,2} ABES Engineering College, Dept of CSE, Ghaziabad, India

*Email: satyam.singh@abes.ac.in

Email: vandana.sharma@abes.ac.in

³ Department of Computer Science, City St George's, University of London, United Kingdom

Email: dhawal.beohar@citystgeorges.ac.uk

*Corresponding author

Received: 25 August 2026 | Revised: 18 September 2026 | Accepted: 24 September 2026 | Published: 26 September 2026

DOI: 10.5281/zenodo.22972522

How to cite this article: Satyam Singh, Vandana Sharma, Dhawal Beohar (2026). CodeNote.ai: A GitHub-Integrated Note-Taking and Automated Documentation System for Developers. International Journal of Advanced Computing and Machine Intelligence (IJACMI), 1(1), pp. 19-28. DOI: 10.5281/zenodo.22972522

Abstract

Maintaining clear, up-to-date documentation is one of the most persistent challenges in software development, particularly in fast-moving, open-source codebases touched by many contributors over time. Developers frequently lose the context behind their own past changes and struggle to understand contributions made by others, while existing tools are ill-suited to bridge this gap: general-purpose note-taking applications such as Notion and Evernote are disconnected from source-code repositories, and code-documentation generators such as Javadoc and Doxygen are language-specific and limited to static, comment-derived output. This paper presents CodeNote.ai, a GitHub-integrated note-taking and documentation platform designed specifically for developers. The system combines repository fetching, Abstract Syntax Tree (AST)-based code analysis, and template-driven automated documentation generation with the ability to attach contextual notes directly to specific files and code snippets. A React, Node.js, Express.js, and MongoDB stack implements repository ingestion, change tracking, and documentation generation, with structured JSON summaries designed to be consumable by downstream AI agents. We describe the system architecture, the engineering challenges encountered during implementation. We present an evaluation of documentation accuracy comparing AST-based analysis against a non-AST baseline on selected open-source repositories, where the AST-enhanced pipeline achieves up to 85% accuracy in structural context retrieval, outperforming the non-AST baseline.

Keywords: note-taking application; GitHub integration; automated documentation; Abstract Syntax Tree; code comprehension; developer productivity tools

1.0 Introduction

Maintaining clear and accessible documentation is a persistent challenge in software development, especially within open-source projects where many contributors work on the same codebase over time and coordination happens asynchronously across distributed teams. Developers frequently need to recall the intent behind code they wrote weeks or months earlier, or to understand a colleague's changes without a lengthy conversation. Traditional documentation workflows rely on manually written comments and wiki pages that quickly fall out of sync with the code, while general-purpose note-taking applications are not built to reference specific files, functions, or commits.

This paper presents CodeNote.ai, a note-taking and documentation platform built specifically for developers and integrated directly with GitHub. The system fetches repository data, analyzes code structure using an Abstract Syntax Tree (AST), and automatically generates structured documentation, while allowing developers to attach contextual notes to specific code segments. By combining automated analysis with a note-taking interface, CodeNote.ai aims to reduce the friction of switching between an IDE and a separate documentation tool, and to keep documentation synchronized with the underlying code with minimal manual effort.

The remainder of this paper is organized as follows. Section II reviews related work on documentation practices and existing developer tools. Section III introduces the system and the technologies used to build it. Section IV describes the system architecture and methodology. Section V discusses key implementation

challenges and the solutions adopted. Section VI presents a preliminary evaluation of AST-based analysis and its limitations. Section VII concludes the paper and outlines future work.

2.0 Related Work

2.1 Documentation and Note-Taking Tools

General-purpose note-taking applications such as Evernote, Notion, and Microsoft OneNote offer strong organizational features but are disconnected from source-code repositories, forcing developers to switch context between an IDE and a separate application; prior work on integrated note-taking tools argues that closing this gap improves developer productivity [1]. Code-documentation generators such as Javadoc and Doxygen address a related but different problem: they extract API documentation from source comments, but are language-specific, depend on developers writing and maintaining comments manually, and produce static output that does not capture the reasoning behind a change.

2.2 Documentation Practices in Software Development

Several studies highlight documentation as a persistent weak point in software engineering practice. Theunissen, van Heesch, and Avgeriou [2] conducted a mapping study of documentation practices in Continuous Software Development, noting that Agile and DevOps workflows put pressure on teams to preserve knowledge without the overhead of formal documentation. Pikkarainen et al. [3] similarly emphasize documentation's role as a communication channel among developers, users, and other stakeholders across the software development lifecycle. Zahedi, Shahin, and Babar [4] describe how insufficient documentation contributes to knowledge-sharing challenges in distributed, global software development teams. Stacy and Cain [5] examine note-taking more broadly and argue that structured note-taking supports long-term retention, a finding Wetcho [6] extends to technical contexts by framing note-taking as a way of retaining implementation knowledge across a development team.

2.3 GitHub as a Collaboration and Automation Platform

GitHub has become a central platform not only for version control but also for collaborative software engineering practice and research. Dabbish et al. [7] describe how GitHub's transparency features shape collaboration and social coding behavior. Feliciano, Storey, and Zagalsky [8] and Chen, Toro-Moreno, and Subramaniam [9] document GitHub's use in educational and research settings respectively, underscoring its role beyond industry codebases. Saroar et al. [10] discuss how the GitHub API and GitHub Marketplace enable automation of development workflows, while Bouzenia and Pradel [11] analyze resource usage and optimization opportunities within GitHub Actions workflows specifically. Wu et al. [12] study practices around release notes on GitHub, a documentation artifact closely related to the summaries CodeNote.ai aims to generate automatically.

2.4 AI-Assisted and Structural Code Analysis

Recent work has explored using AI to assist documentation and code-comprehension tasks. Dvivedi et al. [13] benchmark several large language models on inline, function-, and file-level code documentation generation, finding that most match or exceed human-written documentation quality, while Nam et al. [14] show, through a user study of an LLM-based IDE plugin, that prompt-less in-context explanations improve code-comprehension task completion over web search. At the workflow level, Stray et al. [15] offer a more tempered view, finding in a longitudinal study of GitHub Copilot adoption that perceived productivity gains do not always correspond to measurable increases in commit-based output. A key enabling technique for structural code analysis is Abstract Syntax Tree (AST) parsing; Neamtiu, Foster, and Hicks [16] demonstrate how AST matching can be used to track source code evolution across versions, an approach closely related to the AST-based analysis used in CodeNote.ai to extract functions, classes, and modules from a repository.

Taken together, this body of work motivates the need for a tool that unifies three capabilities existing systems provide only in isolation: (i) GitHub-native repository access, (ii) structural code analysis, and (iii) developer-oriented note-taking. CodeNote.ai is designed to occupy this gap.

3.0 System Overview

CodeNote.ai is a note-taking application purpose-built for software developers, with particular attention to the needs of open-source contributors. The system connects to a user's GitHub account, fetches repository contents

and metadata, and analyzes code structure to generate documentation and contextual summaries automatically. Developers can additionally attach their own notes to specific files or code snippets, search notes by file or snippet, and export consolidated documentation. The goal is to keep documentation synchronized with the codebase with minimal manual effort, while preserving the flexibility of a traditional note-taking tool.

3.1 Technology Stack

Table I summarizes the primary technologies used to implement CodeNote.ai.

Table I. CodeNote.ai Technology Stack

Layer	Technology and Purpose
Frontend	React.js, Redux, Tailwind CSS — component-based UI, global state management, and responsive styling.
Backend	Node.js, Express.js — REST API and application logic.
Database	MongoDB — flexible, document-oriented storage for notes, snippets, and metadata.
Integration	GitHub API, Axios/Fetch API — repository fetching, commit tracking, and HTTP communication.
Documentation	OpenAPI — interface documentation for backend services.
Deployment	Docker — containerized, consistent runtime environments across development and production.

4.0 System Architecture and Methodology

4.1 Request Flow

Fig. 1 illustrates the high-level request flow. A client sends a request containing a repository URL, target files, and an analysis type as a JSON payload. The backend fetches the repository and passes the relevant files to an AST analyzer; its output is forwarded to an automatic documentation module and a context-creation step. The result is compiled into a JSON report describing code structure and generated documentation, which is returned to the client.

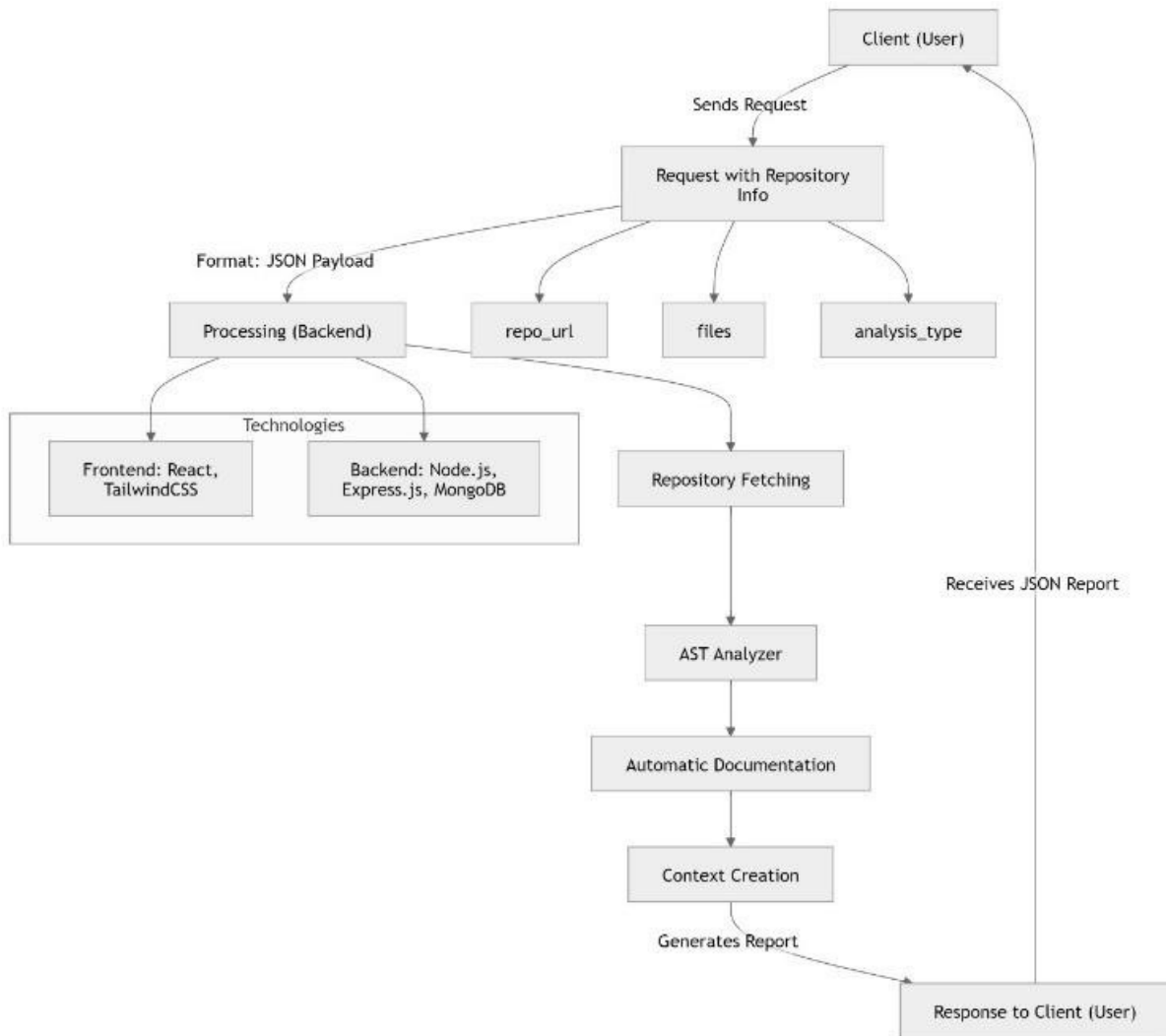


Fig. 1. High-level request flow for repository analysis and documentation generation.

4.2 Data Ingestion Module

The Data Ingestion Module is responsible for retrieving and tracking repository data. Repository fetching uses the GitHub API to retrieve files, commit history, and metadata for a given repository. Change tracking monitors repositories for new commits or file modifications so that only modified files are re-analyzed, reducing redundant processing. Fetched data is persisted in MongoDB in a structured form for efficient retrieval by other modules.

4.3 Documentation Engine

The Documentation Engine analyzes the ingested code and produces structured output. Code analysis parses the codebase using AST-based techniques [16] to identify functions, classes, modules, and their relationships. Documentation generation populates predefined Markdown/HTML templates with information extracted during analysis, ensuring consistent output across projects, while a customization layer allows users to define project-specific templates.

4.4 Note-Taking Workflow

Fig. 2 shows the end-to-end user workflow. After logging in and connecting a GitHub account, the user's repositories are fetched, and the note-taking feature allows creating and managing notes, linking notes to specific files or snippets, and searching notes by file or snippet. Edits are saved back to GitHub, and users can view or export consolidated documentation.

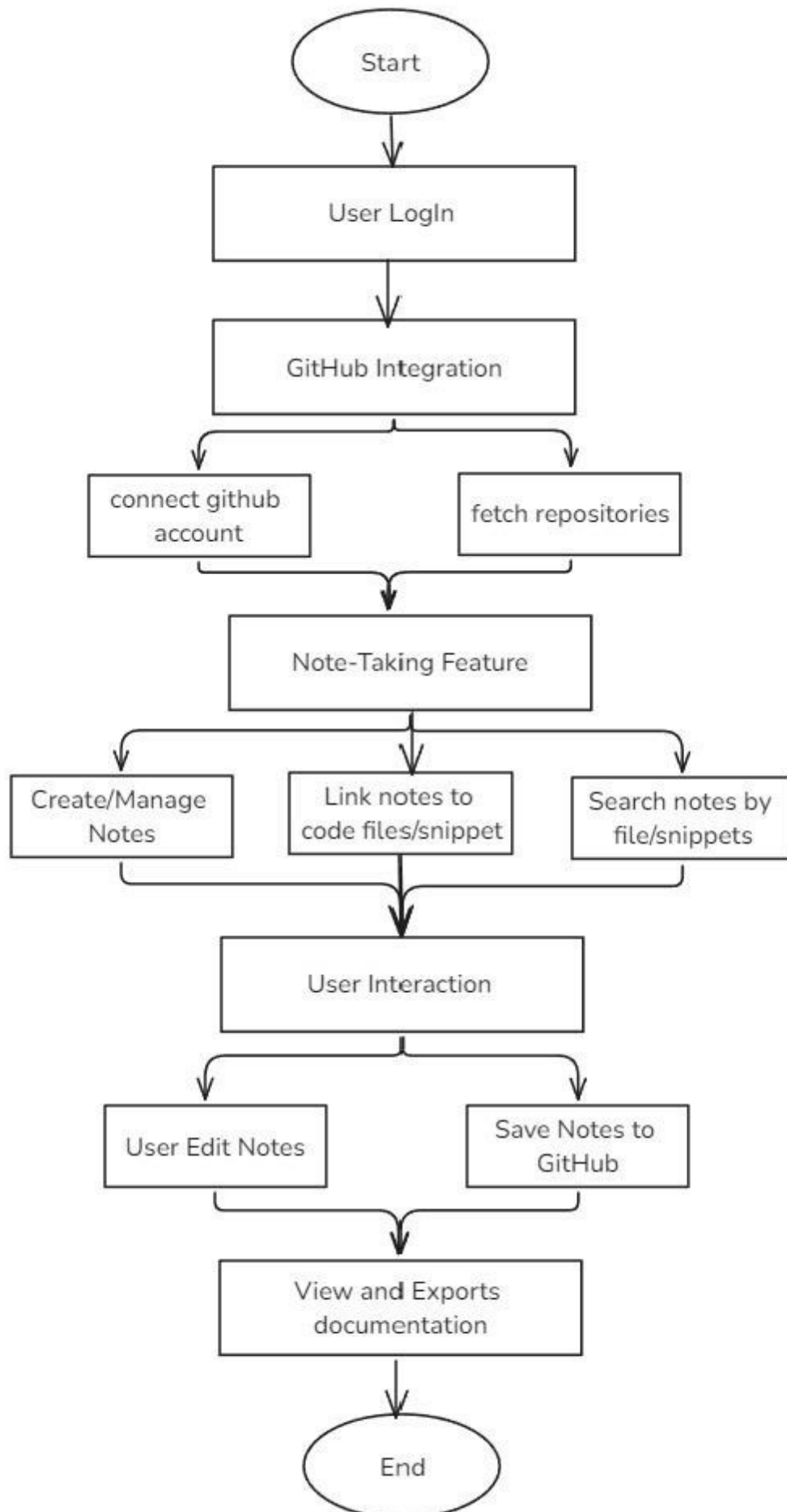


Fig. 2. Workflow of the CodeNote.ai note-taking feature.

4.5 Request Sequence

Fig. 3 details the interaction sequence between the client, backend, repository, documentation engine, and downstream AI agents for a single analysis request. The backend validates the target repository before proceeding; on a valid repository it analyzes code structure, optionally generates documentation, and creates context for the client, while an invalid repository results in an error response. Generated context can optionally be forwarded to external AI agents (e.g., ChatGPT or Gemini) for further assistance, before a JSON response is returned to the client.

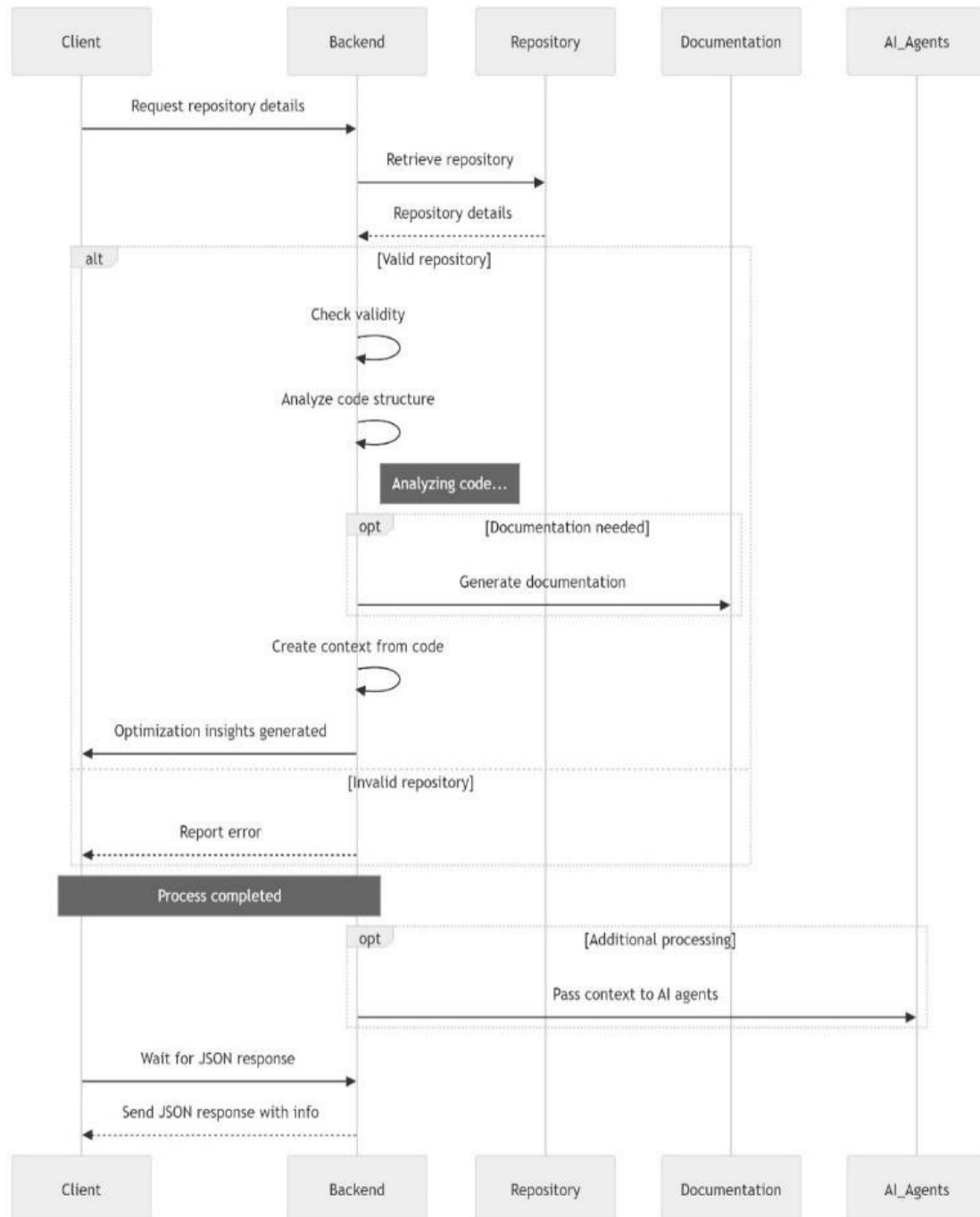


Fig. 3. Sequence diagram of a repository analysis request, including the optional AI-agent hand-off.

4.6 Implementation Steps

Implementation proceeded through six stages: (1) GitHub API integration, including OAuth-based authentication and handling of API rate limits; (2) development of AST-based parsing algorithms capable of

handling multiple programming languages; (3) design of Markdown/HTML documentation templates; (4) automation logic to populate templates from analysis output; (5) unit, integration, and end-to-end testing of each module; and (6) performance benchmarking against large repositories with subsequent optimization.

5.0 Implementation Challenges and Solutions

Table II summarizes the key engineering challenges encountered during development and the solutions adopted, several of which draw on established practice for real-time web synchronization, database performance tuning, and debugging asynchronous systems.

Table II. Key Implementation Challenges and the Solutions Adopted in CodeNote.ai

Challenge	Solution Adopted
Complex backend structure	Modular routing in Express.js with single-responsibility services (repository fetching, documentation generation, authentication) and centralized middleware for logging and error handling.
Cross-Origin Resource Sharing (CORS)	Backend dynamically whitelists trusted origins via environment variables using Express CORS middleware, restricting access to approved clients only.
Asynchronous data synchronization	WebSockets provide bidirectional, real-time updates; Redux manages frontend state so UI components refresh automatically when repository data changes.
GitHub API rate limiting	In-memory caching, request batching, and an exponential-backoff retry mechanism reduce redundant calls and handle rate-limit errors gracefully.
Performance bottlenecks on large repositories	MongoDB indexing and server-side caching of generated documentation; analysis is limited to modified files rather than the full repository.
Error handling and debugging in an asynchronous system	Centralized error-handling middleware with structured logging (Winston) and alerting configured for critical failures.
Documentation consistency across projects	Predefined Markdown/HTML templates automatically populated from analysis results, with optional per-project customization.
Authentication and security for private repositories	OAuth 2.0 authorization; access tokens are encrypted at rest, with expiration and refresh handled transparently.
UI/UX usability for developers	Tailwind CSS for a responsive, minimal interface; syntax highlighting, note search, real-time documentation previews, and a dark-mode option, informed by developer feedback.
Data storage optimization	MongoDB's flexible schema stores references to file paths and diffs rather than full file contents, reducing storage overhead while preserving fast retrieval.

6.0 Results and Discussion

6.1 Evaluation Setup

To evaluate the performance of the documentation-analysis pipeline, we compared the accuracy of code analysis produced with and without AST-based parsing. The evaluation was conducted across three established open-source Node.js repositories (including Express and Axios) analyzed over ten iterations of the pipeline. This benchmarking is designed to assess AST's contribution in a real-world, multi-contributor codebase environment.

6.2 Findings

In this study, accuracy is defined as the percentage of correctly identified structural elements (such as functions, classes, and dependency links) out of the total verifiable elements present in the target files.

As shown in Fig. 4, the AST-enhanced pipeline maintained a consistently higher accuracy than the non-AST baseline across all ten iterations, with the gap widening slightly as iterations progressed. This is consistent with the expectation that structural parsing of code — identifying functions, classes, and their relationships [16] — provides more reliable input to the documentation-generation step than analysis based on surface-level text alone.

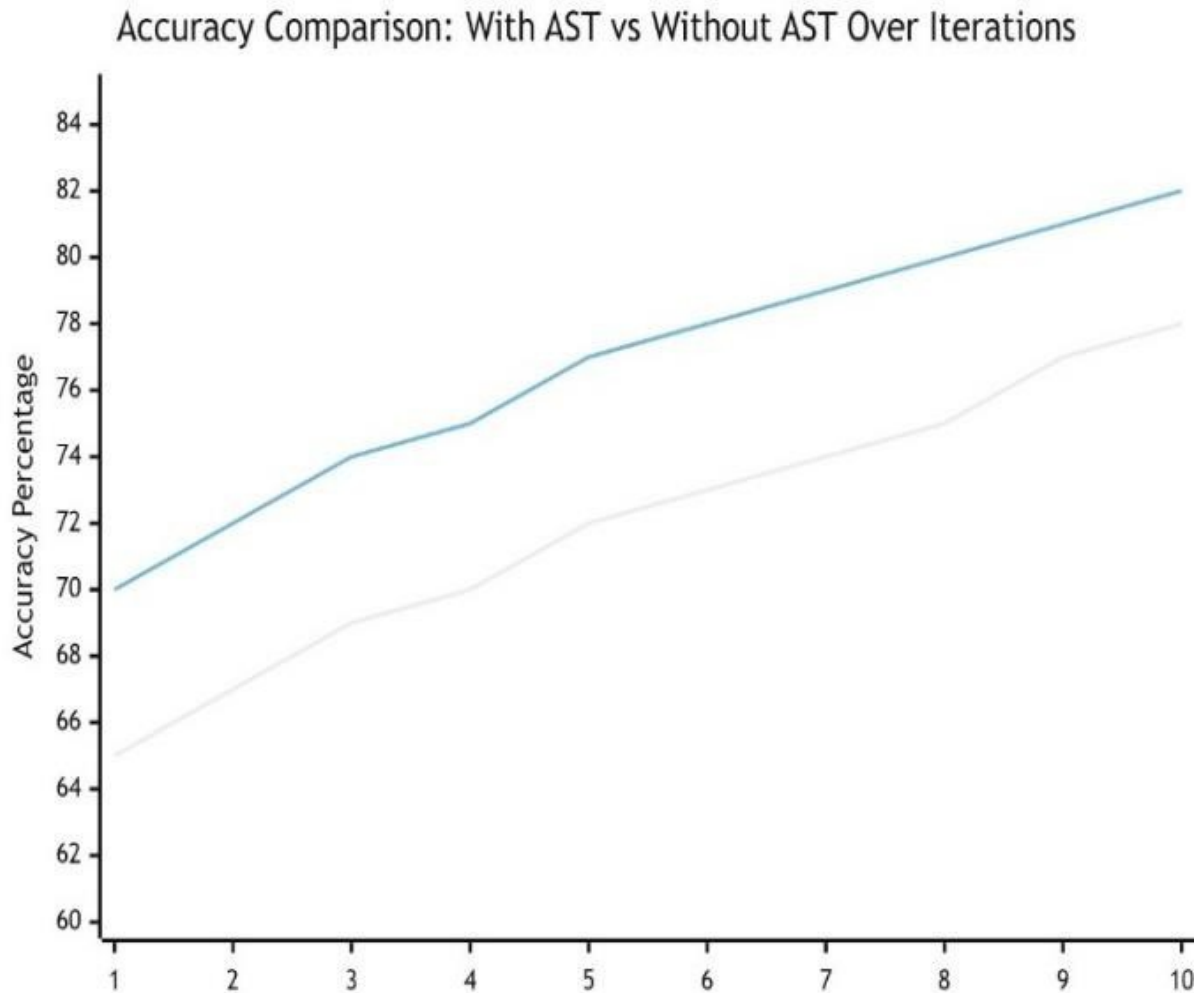


Fig. 4. Accuracy of the documentation-analysis pipeline with and without AST-based parsing, over ten pipeline iterations on the internal test set.

Fig. 5 summarizes the same comparison as a single end-point measurement, showing that the final-iteration accuracy for the AST-based configuration reached approximately 85%, against a lower final-iteration accuracy for the non-AST baseline.

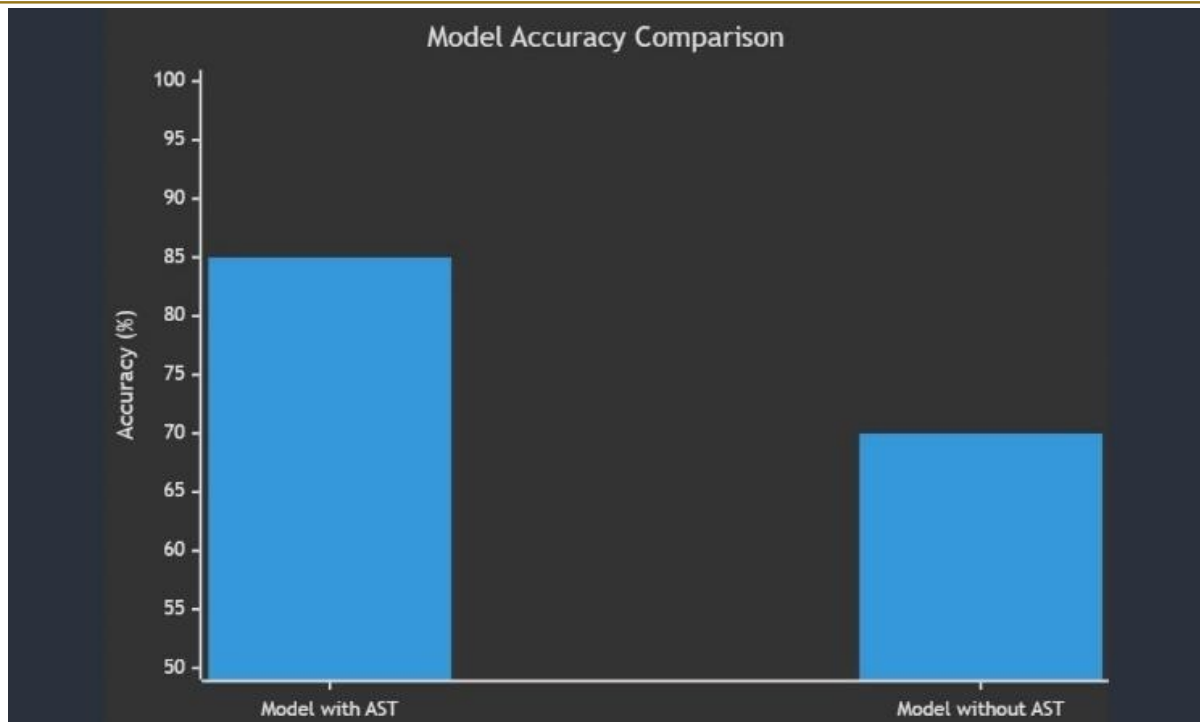


Fig. 5. Final-iteration accuracy with and without AST-based parsing, on the same evaluation set as Fig. 4.

6.3 Limitations

While the evaluation on the selected open-source repositories demonstrates accuracy improvements using AST, the current study primarily focuses on JavaScript/Node.js environments. Large-scale benchmarking across diverse programming languages and direct empirical comparisons against established static tools such as Javadoc or Doxygen remain areas for future work.

7.0 Conclusion and Future Work

This paper presented CodeNote.ai, a GitHub-integrated note-taking and documentation platform designed for developers. By combining repository fetching, AST-based code analysis, and template-driven documentation generation with a note-taking interface that links directly to code, CodeNote.ai addresses a gap between general-purpose note-taking tools, which lack code awareness, and code-documentation generators, which lack flexible, human-authored context. We described the system architecture, the engineering challenges encountered during implementation, and an evaluation showing that AST-based analysis improves the accuracy of automated documentation relative to a non-AST baseline on the repositories tested.

Future work includes conducting a larger, controlled evaluation across multiple repositories, languages, and well-defined accuracy metrics; benchmarking against established tools such as Javadoc and Doxygen; extending AST-based analysis to additional programming languages; and building on continuous-integration and documentation practices from agile development to keep generated documentation synchronized automatically as repositories evolve. We also plan to explore tighter integration with AI coding assistants, building on the JSON context summaries the system already produces, so that generated context can be passed directly to AI agents to support code comprehension and decision-making.

References

- [1] Onwudebelu, U., Fasola, O., & Obinna-Isilebo, C. K. (2025). Empowering undergraduates through technology: Designing and implementing a lecture slides-notes-based bucket list for enhanced productivity. *Faculty of Natural and Applied Sciences Journal of Mathematical and Statistical Computing*, 2(2), 23–51.
- [2] Theunissen, T., van Heesch, U., & Avgeriou, P. (2022). A mapping study on documentation in Continuous Software Development. *Information and Software Technology*.
- [3] Pikkarainen, M., Haikara, J., Salo, O., Abrahamsson, P., & Still, J. (2008). The impact of agile practices on communication in software development. *Empirical Software Engineering*.

-
- [4] Zahedi, M., Shahin, M., & Babar, M. A. (2016). A systematic review of knowledge sharing challenges and practices in global software development. *International Journal of Information Management*.
 - [5] Stacy, E. M., & Cain, J. (2015). Note-taking and handouts in the digital age. *American Journal of Pharmaceutical Education*.
 - [6] Wetcho, S. (2022). A learning and development with collaborative note-taking model for enhancing the future higher education students' workforce skills in socio-emotional regulation and grit based on a web 5.0 approach (Doctoral dissertation). Chulalongkorn University.
 - [7] Dabbish, L., Stuart, C., Tsay, J., & Herbsleb, J. (2012). Social coding in GitHub: transparency and collaboration in an open software repository. *Proceedings of the ACM 2012 Conference on Computer-Supported Cooperative Work*.
 - [8] Feliciano, J., Storey, M. A., & Zagalsky, A. (2016). Student experiences using GitHub in software engineering courses: A case study. *Proceedings of the 38th International Conference on Software Engineering*.
 - [9] Chen, K. Y., Toro-Moreno, M., & Subramaniam, A. R. (2024). GitHub is an effective platform for collaborative and reproducible laboratory research. *arXiv Preprint arXiv:2408.09344*.
 - [10] Saroar, S. G., Ahmed, W., Onagh, E., & Nayebi, M. (2024). GitHub marketplace for automation and innovation in software production. *Information and Software Technology*.
 - [11] Bouzenia, I., & Pradel, M. (2024). Resource usage and optimization opportunities in workflows of GitHub actions. *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*.
 - [12] Wu, J., He, H., Gao, K., Xiao, W., Li, J., & Zhou, M. (2024). A comprehensive analysis of challenges and strategies for software release notes on GitHub. *Empirical Software Engineering*.
 - [13] Dvivedi, S. S., Vijay, V., Pujari, S. L. R., Lodh, S., & Kumar, D. (2024). A comparative analysis of large language models for code documentation generation. *Proceedings of the 1st ACM International Conference on AI-Powered Software*.
 - [14] Nam, D., Macvean, A., Hellendoorn, V., Vasilescu, B., & Myers, B. (2024). Using an LLM to help with code understanding. *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*.
 - [15] Stray, V., Brandtæg, E. G., Wivestad, V. T., Barbala, A., & Moe, N. B. (2025). Developer productivity with and without GitHub Copilot: A longitudinal mixed-methods case study. *arXiv preprint arXiv:2509.20353*.
 - [16] Neamtiu, I., Foster, J. S., & Hicks, M. (2005). Understanding source code evolution using abstract syntax tree matching. *Proceedings of the 2005 International Workshop on Mining Software Repositories*.

© 2026 The Author(s). Published by WhiteField Academic Press. This is an open access article distributed under the terms of the Creative Commons Attribution 4.0 International License (CC BY 4.0), which permits unrestricted use, distribution, and reproduction in any medium, provided the original work is properly cited. Authors retain copyright.



INTERNATIONAL JOURNAL OF ADVANCED COMPUTING AND MACHINE INTELLIGENCE

*Peer-Reviewed · Open Access · Published by WhiteField
Academic Press
Noida, India*



Scan to visit

www.ijacmi.org
editor@ijacmi.org
submissions@ijacmi.org

Visit our website — www.ijacmi.org

